

Student: Manula Manjitha Hindurangala

Student ID: 300390724

Date: March 9, 2026

Work Date/Hours Logs

Date	Hours	Description of work done
Mar 3, 2026	4	Implemented custom DDoS detection rules (Rule1, Rule2, Rule3) in classifier.py with severity based on error budget; enhanced ddos_simulator.py to generate higher load (200 threads, 0.005s delay) for reliable triggering.
Mar 4, 2026	3	Attempted to deploy updated classifier; encountered module reload issue – the running pod continued using the old rule. Debugged by inspecting file contents inside pod and researching Python import caching.
Mar 5, 2026	3	Resolved deployment issue by rebuilding the Docker image with the new classifier and restarting the deployment. Verified that all three DDoS rules were now active.
Mar 6, 2026	2	Tested DDoS detection with the enhanced simulator; confirmed Rule1 fired correctly. Noted that Rule2 and Rule3 require specific traffic patterns (latency spikes, error rate) and documented thresholds for future tuning.
Mar 7, 2026	3	Added brute force and misconfiguration rules (Rule4, Rule5) to classifier.py; created /api/login and /api/broken endpoints in app.py; wrote bruteforce_simulator.py and misconfig_simulator.py; committed all changes.
Mar 9, 2026	1	Created the progress report
Total	16	

Summary Description of Work Done During This Reporting Period

This period focused on completing the **MVP incident classification** for DDoS, brute force, and configuration errors.

Work Completed:

1. DDoS Detection Enhancement

- Refined the DDoS simulator to generate higher load (200 threads, minimal sleep) ensuring reliable triggering of resource-based anomalies.
- Implemented three custom DDoS rules in classifier.py:
 - Rule1 (Resource saturation): request rate >200% AND (CPU >50% OR memory >30%)
 - Rule2 (User impact): request rate >150% AND (latency p95/p99 >100%)
 - Rule3 (Service failure): request rate >150% AND (error rate >200% or absolute >5%)
- Severity is now determined by error rate (P1/P2/P3) following SRE principles.

2. Brute Force & Misconfiguration Detection

- Added two new endpoints to the Flask app: /api/login (always returns 401) and /api/broken (always returns 500).
- Extended the classifier with:
 - Rule4: High authentication failures (auth_failures deviation >200% or absolute >10) → brute force.
 - Rule5: Error rate spike >100% without CPU anomaly → configuration error.
- Created dedicated simulators for brute force and misconfiguration to test these rules.

3. Deployment & Troubleshooting

- Encountered a persistent issue where the updated classifier.py was not being picked up by the running pod (Python module cache).
- Resolved by rebuilding the Docker image and restarting the deployment, ensuring the new code is baked in.
- Verified all three rule families now trigger correctly.

Next Steps:

- Integrate incident display into the React dashboard (Phase 5/6).
- Add persistent storage for incidents.
- Fine-tune thresholds based on observed behavior.

Repo Check-in of Implementation Completed

The following files were added or modified and checked into GitHub:

- backend/src/classifier.py – Added three-tier DDoS rules and later brute-force & misconfiguration rules.
- scripts/ddos_simulator.py – Enhanced to generate higher load for reliable DDoS testing.
- backend/src/app.py – Added /api/login and /api/broken endpoints for attack simulation.
- scripts/bruteforce_simulator.py – New script to generate high authentication failure rate.
- scripts/misconfig_simulator.py – New script to generate 5xx errors without CPU spike.

All code has been tested in the Kubernetes environment; the system now correctly identifies DDoS, brute force, and misconfiguration incidents.

AI Use Section

AI Tool Name	Version, Type Account	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek	DeepSeek Chat, Free Account	Code structure for multi-rule DDoS classifier	Adapted rules to use real Prometheus metrics and incorporated SRE severity logic based on error budget.
DeepSeek	DeepSeek Chat, Free Account	PromQL queries for latency percentiles and error rate	Validated queries against actual data and adjusted time windows for smoother baselines.
DeepSeek	DeepSeek Chat, Free Account	Handling Python module reload in Kubernetes pods	After AI suggested copying files, I realised a full image rebuild was needed; implemented Docker rebuild process.
DeepSeek	DeepSeek Chat, Free Account	Design of brute-force detection using auth_failures	Combined deviation threshold with absolute value to handle near-zero baselines.

AI Tool Name	Version, Type Account	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek	DeepSeek Chat, Free Account	Script structure for brute-force and misconfig simulators	Added realistic delays and endpoint selection to avoid overloading the system during tests.

Appendix: Prompt History

Prompt 1:

"Help me design three rules for DDoS detection using request rate, CPU, memory, latency, and error rate. Each rule should have different confidence levels and severity based on error budget."

Prompt 2:

"Generate a Python script that simulates a high-volume DDoS attack with 200 threads hitting multiple endpoints."

Prompt 3:

"I updated classifier.py in the pod but the detector still uses the old rules. How can I force Python to reload the module without restarting the pod?"

Prompt 4:

"What's the best way to bake the classifier into the Docker image so that new pods always have the latest version?"

Prompt 5:

"How to detect brute force attacks using auth_failures metric? Need to handle cases where baseline is zero."

Prompt 6:

"Create a Flask endpoint that always returns 401 and one that always returns 500, and ensure they increment Prometheus counters."

Prompt 7:

"Write a Python script to simulate brute force login attempts against a /login endpoint with many threads."

Prompt 8:

"Write a script to test misconfiguration by repeatedly hitting a broken endpoint that returns 500, without increasing CPU load."